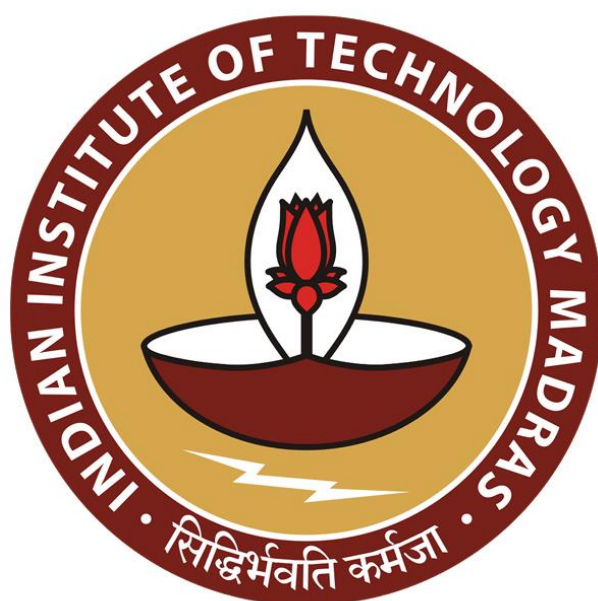




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Higher Order Functions

A higher order function is a function that takes another function as an argument. So, we have not seen higher order functions so far in Java. So, let us see how this whole thing is handled in Java. First of all, where would you need higher order functions?

(Refer Slide Time: 00:29)

IIT Madras BSc Degree

Passing functions

- Recall callbacks
 - `Myclass m` creates a `Timer t`
 - `t` starts running in parallel
 - `t` notifies `m` when the time limit expires

Madhavan Mukund Higher order functions Programming Concepts using Java 2/9

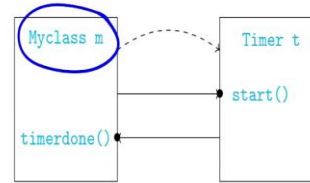
IIT Madras BSc Degree

Passing functions

- Recall callbacks
 - `Myclass m` creates a `Timer t`
 - `t` starts running in parallel
 - `t` notifies `m` when the time limit expires
 - `m` needs to pass `timerdone()` to `t`

Madhavan Mukund Higher order functions Programming Concepts using Java 2/9

- Recall callbacks
 - Myclass m creates a Timer t
 - t starts running in parallel
 - t notifies m when the time limit expires
- m needs to pass timerdone() to t
- Achieved this through an interface



```

public interface Timerowner{
    public abstract void timerdone();
}

public class Myclass
    extends Timerowner{
    ...
}

public class Timer implements Runnable{
    private Timerowner owner;
    ...
    public void start(){
        ...
        owner.timerdone();
    }
}
  
```



Madhavan Mukund

Higher order functions

Programming Concepts using Java 2/9

So, recall our callback example. So, what we said was that a class m, an object of class m, starts off a timer t in parallel. So, this starts off in parallel. And the goal is that the class m should continue with this computation and the timer t being a timer is given some time to run out. And when the time runs out, it must call back. So, this is the idea of callback, it must be notifying, it should notify the original object time that the time has expired.

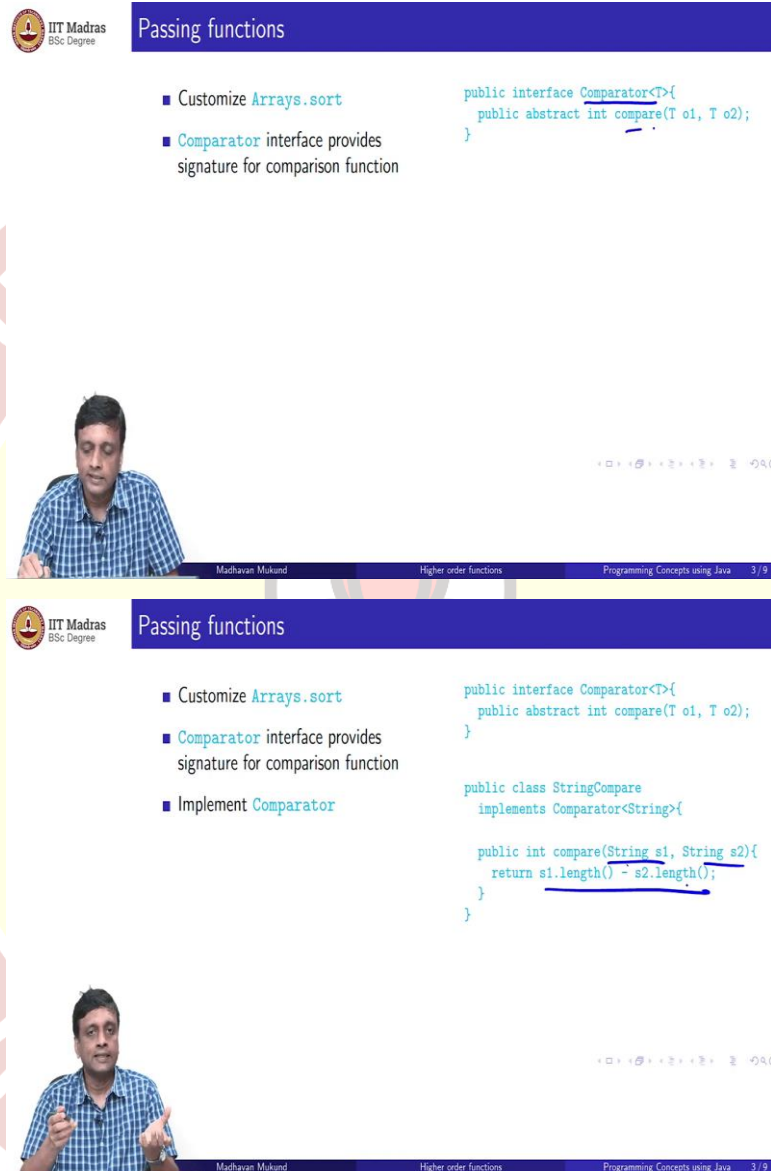
So, the issue that we faced was how to pass this information. So we needed to pass this function from here to here. So, this is the whole purpose of this higher order functions, you need to pass a function from some part of the code to some other part of the code. So, what Java and in general object-oriented programming says about passing functions is you do not pass functions directly, you pass them by encapsulating them in an interface.

So, we do not pass the function, but we rather we pass an object on which we can call the function. So, we created this interface called Timerowner. And this interface guarantees that any object that implements Timerowner has associated with the function timer done. So, then the class that creates this timer, extends Timerowner. So, it will have an implementation of this timerdone guaranteed.

And now the timer class, will store an object of this type, which is which it receives at the time of starting. So, it is a part of the constructor of that object, if you remember, we pass the name of the owner. So, when it is done, it can take this owner and knowing that it implements this interface, it knows this function is there.

So, the function is not passed directly, it is rather passed through an object, which knows how to implement the function, that is how this higher order function passing is done in a typical object-oriented setting, and that is how it is done in Java.

(Refer Slide Time: 02:31)



IIT Madras
BSc Degree

Passing functions

- Customize `Arrays.sort`
- `Comparator` interface provides signature for comparison function
- Implement `Comparator`

```
public interface Comparator<T>{  
    public abstract int compare(T o1, T o2);  
}
```

```
public class StringCompare  
    implements Comparator<String>{  
  
    public int compare(String s1, String s2){  
        return s1.length() - s2.length();  
    }  
}
```

Madhavan Mukund Higher order functions Programming Concepts using Java 3/9

- Customize `Arrays.sort`
- `Comparator` interface provides signature for comparison function
- Implement `Comparator`
- Pass to `Arrays.sort`

```
public interface Comparator<T>{
    public abstract int compare(T o1, T o2);
}

public class StringCompare
    implements Comparator<String>{

    public int compare(String s1, String s2){
        return s1.length() - s2.length();
    }
}

String[] strarr = new ...;
Arrays.sort(strarr, StringCompare);
```



Madhavan Mukund

Higher order functions

Programming Concepts using Java

3/9

So, let us look at another example. So, we have this static method, rather a family of static methods called sort in, in the Arrays class, which allows us to sort many different things. So, it is static and the distinguishing thing between saying sorting an array of doubles and floats and all that is done through this overloading of the signature.

So, now we can customize this by passing a function which tells `Arrays.sort` how to compare elements, because ultimately what is sorting, but comparing elements and putting the smaller one before the bigger one, say it is in ascending order. So, for this there is an interface again, because we need to pass a function what we need to do is tell `Arrays.sort`, here is the function that you should use to compare the elements in the array.

And since we cannot pass a function, what this `Arrays.sort` expects is an object which implements a `Comparator` interface for that particular type. So, if I have an array of strings, then I must implement comparator for strings. If I have an array of doubles, I must do for double and so on. And what is the `Comparator` function do? It guarantees that there is a function which takes two objects of that type, and then returns a negative 0 or a positive depending on whether the first object is less than equal to or smaller bigger than the second object.

So, now, if I want to now, actually use this, in order to sort a particular array. So I first have to define a class which implements this comparator. So, for instance, I could write a string compare thing which implements this, and what it does is it takes two strings, and then it just say, for example, compares them by length. So, if the first string is smaller in length in the second string, you will get a negative number if it is equal to you will get a 0, if it is bigger than you will get a positive number which is what comparator expects.

Of course, the comparator interface like any other interface can only tell you the signature of the function, it cannot tell you that the meaning has to be this that is up to you to figure out the documentation and get it but at least syntactically this works.

So, now I can create a string array and I can say please sort this string array using this particular implementation of the comparator interface and an Arrays.sort will work. So, this is the object-oriented solution to passing functions to encapsulate it in an interface and then you passed.

(Refer Slide Time: 04:50)

Functional interfaces

- Interfaces that define a single function are called **functional interfaces**
 - **Comparator, TimerOwner**
- How can we directly pass the required function?
- In Python, function names are similar to variable names
 - Define a function
 - Pass it as an argument to another function
 - **map** is a **higher order function**

```
public interface Comparator<T>{
    public abstract int compare(T o1, T o2);
}

public interface TimerOwner{
    public abstract void timerdone();
}
```

```
def square(x):
    return(x*x)

l = list(map(square, range(100)))
```

Madhavan Mukund Higher order functions Programming Concepts using Java 4/9

So, these particular interfaces that we just looked at comparator and this Timerowner, which is actually something that we invented. So, this, these interfaces are actually very specific kinds of interfaces that the only thing that they have their only job is to encapsulate this function. So, they are also called in Java terminology a functional interface. So, this is an interface which consists largely of one function that needs to, it actually has other things.

So, comparator has some static functions and are defined in it. But as such implementing comparator means implementing one function implementing Timerowner the way which is something we defined involves implementing that notifying the timer that that timerdone function. So, these are called functional interfaces. And the question is, can we now if our only goal is to pass an object of this type to implement that one function that this interface requires, then can we get rid of that wrapper.

So, we are really taking one function, putting it inside a box and passing the box. But the box is guaranteed to have only one function. So, there is no box, it is like taking a box and putting

one thing into it, might as well pass at one thing. Or it is like taking a box and putting it inside a bigger box. If you might have many boxes inside a bigger box, it makes sense. You have many functions inside an object, it makes sense. You have one function inside an object, can we just pass the function itself. That is what we are asking.

So, just to remind you, how this is done in other languages, in Python, for example, the name of a function is equivalent, in some sense to the name of anything else. So, what is the function take in Python, it takes some arguments, and how do you provide the arguments used, you populate the arguments when you call it by providing the names of the arguments that you want to pass. So, in Python, a function name is has the same status as any other names.

So, you can define for instance, a function square, which takes x and squares it. And now, map is a function which takes a list and applies some function to every element in that list. So, here is a list or a sequence at least, range from 0 to 99. And now, I want to square every number in that list. So, I just pass the name square, and this is what we call a higher order function. So, it is a function. So, map itself is a function. And one of its arguments is another function.

So, a higher order function is just a kind of fancy way of saying a function, which takes another function argument, and it is not a very esoteric thing, or it is not a very exotic thing, it is quite normal, I want to take this list, and I want to apply this transformation to every element in this list. So, I need, I need to know how to describe that transformation.

And Java now has no way of doing that. So, this is what happening, we are doing in sort also, we are saying, I want to take this array, and I want to apply this comparison function to sort this array. But I have no way to pass that comparison function now as a function without putting it into this interface and then passing the object.

(Refer Slide Time: 07:54)



Lambda expressions

- Lambda expressions denote anonymous functions

- (Parameters) → Body

- Return value and type are implicit

```
(String s1, String s2) ->  
s1.length() - s2.length()
```



Madhavan Mukund

Higher order functions

Programming Concepts using Java

5/9



Lambda expressions

- Lambda expressions denote anonymous functions

- (Parameters) → Body

- Return value and type are implicit

- From λ -calculus (Alonzo Church)

- Foundational model for computing, parallel to Alan Turing's machines
- Basis for functional programming: Lisp, Scheme, ML, Haskell, ...

```
(String s1, String s2) ->  
s1.length() - s2.length()
```



Madhavan Mukund

Higher order functions

Programming Concepts using Java

5/9



Lambda expressions

- Lambda expressions denote anonymous functions

- (Parameters) → Body

- Return value and type are implicit

- From λ -calculus (Alonzo Church)

- Foundational model for computing, parallel to Alan Turing's machines
- Basis for functional programming: Lisp, Scheme, ML, Haskell, ...

- Substitute wherever a functional interface is specified

```
(String s1, String s2) ->  
s1.length() - s2.length()
```

```
String[] strarr = new ...;  
Arrays.sort(strarr,  
    (String s1, String s2) ->  
    s1.length() - s2.length());
```



Madhavan Mukund

Higher order functions

Programming Concepts using Java

5/9

So, finally, after some evolution, Java decided to allow this. So initially, Java did not have this kind of function definition capacity, where you can pass it on directly. But now Java has something called a lambda expression. So, a lambda expression is essentially an anonymous function. It is a function without a name.

So, what do we have in a function? We have three things. We have the name of the function, we have the types of the arguments, the sequence of arguments, and we have the type of the return. And of course, we have the body of the function, which takes the arguments and does some computation to it, actually we are technically four things.

So, a lambda function, first of all eliminates the name. So, it is anonymous, there is no name. So, it takes two parts, the parameters and the body and it separates them with this arrow. So, this is kind of notation, which is borrowed from mathematics. So, just, we just saw that minus greater than needs to be thought of as an arrow. So, it takes a parameter.

So, normally, when you pass parameters to a function, it is inside brackets, it is a list of parameters, a sequence of parameters inside brackets. So, you keep the bracket and you pass them, and then you have a body, which is how to compute. And now the fourth thing is the return value, and the type of the return value, and that becomes implicit.

So, let us look at an example. So, here is that function that we were writing earlier, to compare two strings by length. So, what this says is that it takes two arguments `s1` and `s2` and returns the difference `s1` length minus `s2` length. So, there is no explicit return statement, that is the first step, just because this expression evaluates to a value given the arguments `s1` and `s2`, `s1.length()-s2.length()` provides me a value and that is implicitly the return thing.

So, the return value is implicitly computed by the expression that you provide. And the type is also an implicit, I am not saying that this is going to be an `int`, but you should know that `length` gives an `int` and therefore, there is a certain amount of type inference going on here. From the other hand, I need to tell you, that `s1` and `s2` are strings.

So, I provide the types of the arguments. I provide an expression involving them and the return value is implicitly the value computed by this expression and the return type is a type of that expression. So, this is called a lambda expression. So, you might wonder why lambda? So, lambda is a Greek letter. And the lambda in this terminology comes from a mathematical theory called the lambda calculus.

So, the lambda calculus was invented by a very famous logician called Alonzo Church about 100 years ago. And the importance of the lambda calculus is that it is one of the foundations of computing, so, it looks if you read the lambda calculus and you read it in a book, it will look very strange, but actually, it provides a model of computing.

So, some of you may have heard of Alan Turing. So, Alan Turing also invented an abstract model of computing which was more like something which actually operated with something moved here, move there. So, he called it a machine. So, Alan Turing had a kind of operational machine-oriented way of doing things and lambda calculus is a different style of computation, which we are not going to discuss.

But the importance of lambda calculus cannot be understated because lambda calculus is the heart of very many functional programming languages, which are used in a wide variety of things. So, starting with the language Lisp, which came up in the 1950s and 60s, to its more modern version called Scheme.

And then, there are even more modern languages called ML, and Haskell and so on, all of these are fundamentally based on the lambda calculus, and the lambda calculus is essentially built out of such expressions. So, that is why these expressions in Java are also called lambda expressions. So, this is just a historical aside, it has nothing to do with how you write the expression, but as to know why it is called a lambda expression.

So, the interesting thing for us is that wherever we have a functional interface, for example, that comparator interface wherever a function expects a functional interface object, you can instead pass a corresponding lambda expression of the given type. So, remember this `Arrays.sort`. So, `arrays dot sort`, we had to write this string comparator object, which extended this comparator of string, and then we have to pass that object.

Now instead, we can directly pass this function, which is effectively doing the same thing as the length comparator that we wrote earlier. So, now `Arrays.sort` will take as its first argument, the array and the second argument, it is taking this expression. So, this expression is just a verbatim copy of that expression.

So, I am just consuming providing the code. And providing the code for the function as an argument. I am not providing a name for the function, just providing the code. So, it is an anonymous function, I am just providing the code. So, it would be like in Python, I could write a block of code in an argument, which I cannot do. So, in Python, you have to put it as a

name and pass it but here, you actually write it as this funny type of expression and pass it so it has the body and the parameters, and the rest of it is all inferred.

(Refer Slide Time: 12:53)

Lambda expressions

- **Lambda expressions** denote anonymous functions
 - (Parameters) → Body
 - Return value and type are implicit
- From λ -calculus (Alonzo Church)
 - Foundational model for computing, parallel to Alan Turing's machines
 - Basis for functional programming: Lisp, Scheme, ML, Haskell, ...
- Substitute wherever a functional interface is specified
- Limited type inference is also possible
 - Java infers `s1` and `s2` are `String`

```
(String s1, String s2) ->
s1.length() - s2.length()

String[] strarr = new ...;
Arrays.sort(strarr,
    (String s1, String s2) ->
    s1.length() - s2.length());

String[] strarr = new ...;
Arrays.sort(strarr,
    (s1, s2) ->
    s1.length() - s2.length());
```

Madhavan Mukund Higher order functions Programming Concepts using Java 5/9

So, if you recall, Java can do some amount of type inference. So, here, it can figure out that okay, this is a string array. So, based on that, I am using this arrays dot sort for strings and I know that in a string array, I am going to look for a comparator of the type comparator string. And therefore, this function that passes here must implement that compare function for comparator of string, therefore, it is two arguments must both be strings, and therefore I do not have to worry about writing this, I just write `s1, s2`.

So, this is a very sort of special case of this type inference. Earlier, we said the type inference will work for local variables defined inside a conventional function method inside an object, a class definition. But here in this lambda expression, if from the context, the types of the values are clear, then Java will allow you to go ahead so you can simplify this thing, if you are not sure, you can always put the type thing and nothing will go wrong, in fact, it will always be correct.

Whereas, here if you do the wrong thing, then Java might complain, cannot infer a type. So, you can skip the type, type definition of the arguments if it is clear from the context. But the main thing is that it is always going to be allowed only when there are functional interfaces.

(Refer Slide Time: 14:05)



Lambda expressions

- More complicated function body can be defined as a block

```
(String s1, String s2) -> {  
    if s1.length() < s2.length()  
        return -1;  
    else if s1.length() > s2.length()  
        return 1;  
    else  
        return 0;  
}
```

$s1.length() - s2.length()$
 $n_1 - n_2$



Madhavan Mukund

Higher order functions

Programming Concepts using Java 6/9



Lambda expressions

- More complicated function body can be defined as a block
- Note that the function is anonymous only for the caller
- The function that receives the lambda expression still needs to use a functional interface for the parameter type

```
(String s1, String s2) -> {  
    if s1.length() < s2.length()  
        return -1;  
    else if s1.length() > s2.length()  
        return 1;  
    else  
        return 0;  
}
```

```
public static <T> void  
Arrays.sort(T[] a, Comparator<T> c)
```

- Inside `Arrays.sort()`, refer to the function by the name `compare()` defined in the `Comparator` interface



Madhavan Mukund

Higher order functions

Programming Concepts using Java 6/9

So, it looks like you can only write simple functions. No, you can write anything you want. So, for instance, if you wanted to the same function, if you want to write it out in elaborate detail. So, instead of just saying length this length of the cipher string minus the segment length of the second string, you could say well, if the first thing is shorter than the second string, then return a minus 1, bigger return upon else return 0.

Now, here you have to put return because this complicated if statement, so return a value. So, the if statement on its own does not evaluate to anything, earlier when I just wrote `s1.length() - s2.length()`. So, this was an arithmetic expression, this gave me n_1 , this gave me n_2 and I got n_1 minus n_2 which is an integer and therefore I did not have to write a return statement. So, if

the, if the expression of the lambda expression, the expression part, the body evaluates to a constant, then you do not have to write a return.

But in order to generate that thing, you might have to put a return. And that is what is happening here. But the whole thing is that by just putting everything in a brace, I can put an arbitrary block of code. So, this lambda that I pass, can be as complicated as I want. You do not have to restrict yourself to a one-line function as we did before.

So, this is an anonymous function, but it is kind of asymmetric when I call something that expects a functional interface, I can pass an anonymous function without naming it creating an object all that. From the other side, there is no notion I cannot write a function which takes an anonymous function as an input directly the only way I can write a function is to actually use interface, so the other side the person or the code that is receiving this thing must be written in the conventional sense.

So, here in this, in this arrays dot sort, it will take an array of some type and it will take a comparator over comparator type. So, this is how the code will look from the point of view of the function that consumes the lambda expression, the function that consumes a lambda expression does not know what is coming as a lambda expression, it only knows that it is something which implements a functional interface.

And is willing to take in place of object of that functional interface type. It is willing to take a lambda expression. So, this is something important to know there is no direct way to write something which you can generate lambda expressions and pass them. But you cannot write code which accepts lambda expressions directly, you have to write this interface at your end.

(Refer Slide Time: 16:30)



Passing named functions

- If the lambda expression consists of a single function call, we can pass that function by name
 - Method reference
- We saw an example with adding entries to a `Map` object
 - Here `sum` is a static method in `Integer`

`Map<String, Integer> scores = ...;
scores.merge(bat, newscore, Integer::sum);`

Handwritten notes: "new" above "newscore", "ac. score" above "Integer::sum", and "Name" with arrows pointing to "Integer::sum" and "Integer" in the code.



Madhavan Mukund

Higher order functions

Programming Concepts using Java

7/9



Passing named functions

- If the lambda expression consists of a single function call, we can pass that function by name
 - Method reference
- We saw an example with adding entries to a `Map` object
 - Here `sum` is a static method in `Integer`
- Here is the corresponding expression, assuming type inference

`Map<String, Integer> scores = ...;
scores.merge(bat, newscore, Integer::sum);`

`(i,j) -> Integer::sum(i,j)`



Madhavan Mukund

Higher order functions

Programming Concepts using Java

7/9



Passing named functions

- If the lambda expression consists of a single function call, we can pass that function by name
 - Method reference
- We saw an example with adding entries to a `Map` object
 - Here `sum` is a static method in `Integer`
- Here is the corresponding expression, assuming type inference
- Expression should call a function, and nothing else — this expression cannot be replaced by a method reference

`Map<String, Integer> scores = ...;
scores.merge(bat, newscore, Integer::sum);`

`(i,j) -> Integer::sum(i,j)`

`(i,j) -> Integer::sum(i,j) > 0`



Madhavan Mukund

Higher order functions

Programming Concepts using Java

7/9

Now, we said that we are passing the entire body of the function. And, we said that in Python, actually, you write the name of the function after defining it and pass the name of the function. So, in certain situations in Java, also, you can pass the name of the function, and this is called method reference.

So, you pass a name of a function, which is a method. So, instead of passing the function itself as a direct lambda expression, you might want to give a function which is already defined somewhere else to be something you have defined or be something that somebody else has defined.

So, for example, we had already seen this, if you remember, we talked about HashMaps. we said that one of the usual problems with a HashMap is when you are trying to increment say a counter associated with the key whether it is the first time or not. So, we had provided many different ways to do it. But one of the ways we did it was to use this merge function.

So, supposing we have a map, which maps strings to integers. So maybe this is scores of say cricket players. So, the first, the key is the name, and this is the accumulated score. So, now when I see a new score for a cricket player, I want to add it, but I see the cricket player for the first time, I want to set it to the current score.

So, one of the ways was to use this merge thing. So, this is the name, this is a string and this is the new score, and there is an implicit value associated with this key before. So, what this merge function says is that if there is a key, then use the sum function defined in the integer class to combine the current value with the new value.

So, this is a function, which we are passing by reference. So, this is a method reference, it is a static function in integer. So, I could instead now pass an explicit function here. So, I can say, take a function, which takes two integers *i* and *j*, and computes their sum using the static integer sum class. So, this is an explicit function, combining *i* and *j*. So, in such a case.

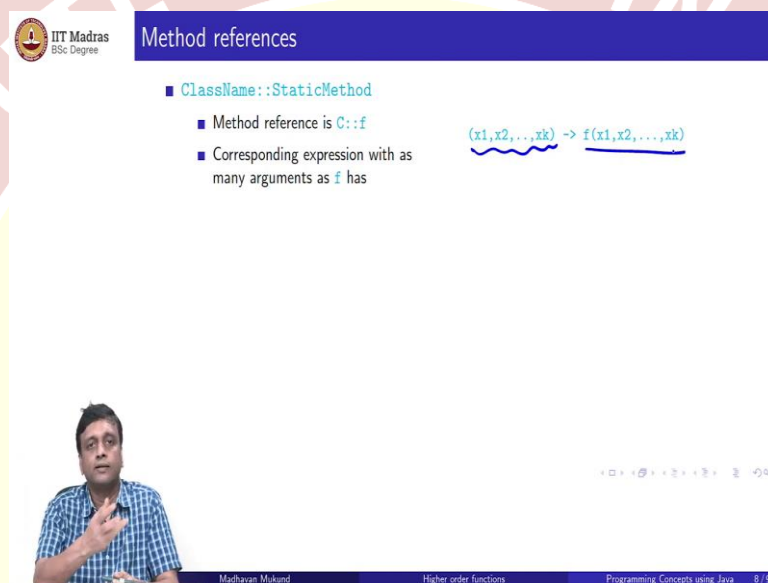
So, if the function consists of a single function call, this is the important thing, the only thing that we do with *i* and *j* is to apply some other function to it and use the value of that function as the return value of this lambda expression, then I can replace it by the name of the function itself, because there is no ambiguity, what needs to be done.

If on the other hand, if you do something with it. For example, supposing I want to return a Boolean, which checks whether the sum is positive. Now, it is no longer just a call to a function, it is a call to a function followed by some check or some additional computation

involving the value returned by that function. In such a case, I have to write the function, I cannot replace the expression by a call to some because it is not a call to some, it is a call to some plus something more.

So, if I am doing something more than just calling the function, I cannot pass the function by name, I have to write an explicit expression which uses that function and combines it in whatever they want. But, if my only goal is to pass an expression that calls another function, I can give you the name of that function be done. So, that is called a method reference.

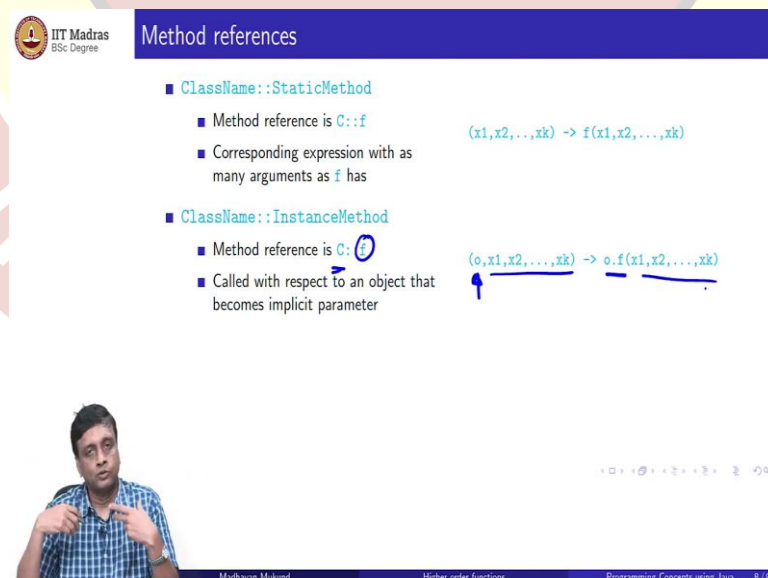
(Refer Slide Time: 19:37)



The slide is titled "Method references" and features the IIT Madras logo in the top left corner. It contains the following content:

- **ClassName::StaticMethod**
 - Method reference is `C::f`
 - Corresponding expression with as many arguments as `f` has

Handwritten notes in blue ink show the transformation: $(x_1, x_2, \dots, x_k) \rightarrow f(x_1, x_2, \dots, x_k)$. The slide also includes a small video feed of the presenter, Madhavan Mukund, and a footer with the text "Higher order functions" and "Programming Concepts using Java 8/9".



The slide is titled "Method references" and features the IIT Madras logo in the top left corner. It contains the following content:

- **ClassName::StaticMethod**
 - Method reference is `C::f`
 - Corresponding expression with as many arguments as `f` has
- **ClassName::InstanceMethod**
 - Method reference is `C::f`
 - Called with respect to an object that becomes implicit parameter

Handwritten notes in blue ink show the transformation: $(o, x_1, x_2, \dots, x_k) \rightarrow o.f(x_1, x_2, \dots, x_k)$. The slide also includes a small video feed of the presenter, Madhavan Mukund, and a footer with the text "Higher order functions" and "Programming Concepts using Java 8/9".

Method references

■ `ClassName::StaticMethod`

- Method reference is `C::f` $(x_1, x_2, \dots, x_k) \rightarrow f(x_1, x_2, \dots, x_k)$
- Corresponding expression with as many arguments as `f` has

■ `ClassName::InstanceMethod`

- Method reference is `C::f` $(o, x_1, x_2, \dots, x_k) \rightarrow o.f(x_1, x_2, \dots, x_k)$
- Called with respect to an object that becomes implicit parameter

■ `object::InstanceMethod`

- Method reference is `o::f` $(x_1, x_2, \dots, x_k) \rightarrow o.f(x_1, x_2, \dots, x_k)$
- Arguments are passed to `o.f`



Method references

■ `ClassName::StaticMethod`

- Method reference is `C::f` $(x_1, x_2, \dots, x_k) \rightarrow f(x_1, x_2, \dots, x_k)$
- Corresponding expression with as many arguments as `f` has

■ `ClassName::InstanceMethod`

- Method reference is `C::f` $(o, x_1, x_2, \dots, x_k) \rightarrow o.f(x_1, x_2, \dots, x_k)$
- Called with respect to an object that becomes implicit parameter

■ `object::InstanceMethod`

- Method reference is `o::f` $(x_1, x_2, \dots, x_k) \rightarrow o.f(x_1, x_2, \dots, x_k)$
- Arguments are passed to `o.f`
- Can also pass references to constructors



So, the kind that we have seen in the previous example is one where I have a class name, namely integer and I have a static method namely some defined there. So, effectively, what will happen is that in the given the context of however, that thing is implemented, it will take some arguments which come out and apply this function `f` to those arguments.

So, in the case of the sum thing, it requires two arguments `n1` and `n2`. And so, the `n1` will be the current value of the hash key and `n2` will be the argument that has applied to the merge function. And these two values will be provided as arguments to the int, the integer add, or the integer sum and given back. So that is one way.

So, there is, it is very clear is effectively a static method in a class is nothing like, nothing more than a normal function in another language like C or anything else, or Python, it is just a function that exists, it is static so, it is always there. And it has to be defined somewhere, so it

is in the class, but otherwise, it has an expectation of what arguments it has been given. And Java will figure out which arguments are to be given based on the context in which you are calling it. So, that code will appropriately pass the arguments to this function.

Now, you could also pass a function associated with a particular object of a particular type. So, now, this would make sense if what you are going to do with this is apply to some object at the other end. So, you pass a method `f`, which is defined in a class `C`. So, what is going to happen is that you will find in the way it is being used, you will find an object of that class.

So, you will apply `f` in the context of `o`, because remember, this is an instance method. So, it can only be invoked on an instance. So, you say `o`, and you say, `o.f`, and the remaining arguments are passed to this. So, you can, you do not restrict yourself to static methods and classes, you can actually have these regular methods and define it in the context of a particular object. And the most elaborate thing is that you can actually pass an object.

So, here I am saying, we pass a function defined with respect to a class, and the object comes out of whatever you are applying it to, and you provide you a context to apply it here, I give you the object and the method saying apply your function with respect to this object, and then you do `o.f`. So, the question is; whether this `o` comes from implicitly or comes explicitly? So, these are all different ways in which I can call this thing.

So, these are all applicable, remember, only when we have a lambda expression, whose only job is to call the function and do nothing more. But that function that you call can be of any of these types. You can also pass for example, constructors and so on. So, these are more elaborate things you can do with these method references.

So, there are constructor references. So, you can look it up. But it is important just to remember that the whole goal of this is to pass a function without passing an object. So, it is just a simplification in some situations of what you want to do.

(Refer Slide Time: 22:41)



Summary

- Many languages support higher-order functions
 - Passing a function as an argument to another function
- In object-oriented programming, this is achieved using interfaces
 - Encapsulate the function to be passed as an object
- Java allows functions to be passed directly in place of functional interfaces
 - Interface consists of a single function
- Lambda expressions describe anonymous functions
 - Cannot pass lambda expressions in general
 - Only when the argument is a functional interface
- Can pass a method reference if the lambda expression consists of a single function call



Madhavan Mukund

Higher order functions

Programming Concepts using Java 9 / 9

So, to summarize, higher order functions are actually very important things. Many situations require one function to use another function. A typical example is a comparison in a sort, when, or what we saw here, which is, if I am called, and I am asked to call back something else, and the callback function has to be given to me. So, like we saw with the timer object.

So, passing a function to another function as an argument is quite a standard thing. And it is allowed in many languages, including Python. But typically, object-oriented programming tries to avoid this, because functions in general do not exist in isolation in objects, they always exist as part of a class.

So, the way that we simulate this normally in object-oriented programming in general, and in Java, in particular, is to define this interface, with capability. A function is associated with the capability the ability to call the function. So, that is captured through an interface. And then we pass an object, which implements that interface with the guarantee that that object has a named function that we know whose name we know, which we can call.

So, Java has relaxed this rule, by allowing functional interfaces to be replaced by calls which contain this lambda expression. So, lambda expression is an explicit representation of a function without a name, it is an anonymous representation. It just talks about the arguments to the function and what to do with these arguments.

You cannot pass these around in general, and you cannot just freely pass around these, you can only pass it when the particular argument you are trying to replace is a functional interface. And remember, a function interface is an interface which consists of exactly one

function that needs to be implemented. So, it is a very limited kind of interface. But this is the kind of thing we see when you use comparator and so on.

And in particular, if the, if the further simplification is that the function, the lambda expression you are passing, consists of only evaluating some particular function once and nothing more, then that lambda expression itself can be replaced by a call to the function, so you do not have to encapsulate a function call inside a function call and pass it, you can just pass the original function. So, that is what we saw in terms of method references.

